

Education

Beihai University (985), Artificial Intelligence, Master's	2026.09 – 2029.06
Donghai University, Computer Science and Technology, Bachelor's	2022.09 – 2026.06
• Academic Record: Passed both CET-4 and CET-6; as an undergraduate, won provincial silver medals in ICPC and CCPC, the National Scholarship, and the second prize (national level) in the Higher Education Press Cup Mathematical Modeling Contest; available for internships of six months or longer.	

Professional Skills

- **LLM Principles & Reasoning Paradigms:** Proficient in Prompt Engineering, CoT, ReAct, Plan-and-Execute, Reflection and other reasoning and Agent design paradigms
- **Agent Development:** Proficient in LangChain, LangGraph and the OpenAI SDK; experienced in Tool Calling, Function Calling, MCP tool integration, multi-turn conversation memory, state management, task orchestration, multi-agent collaboration and Human-in-the-loop development
- **RAG & Vector Databases:** Proficient in full-pipeline RAG development, including document parsing, semantic chunking, parent-child chunking, Embedding, vector retrieval, keyword retrieval, hybrid recall, reranking and answer evaluation; familiar with the Milvus vector database
- **Engineering & AI Development Tools:** Proficient in Python, FastAPI, etc.; familiar with AI coding tools such as Codex and Doubao, and able to use AI to assist with code generation, debugging, refactoring, testing and engineering iteration

Project Experience

Geek Select · Intelligent Digital-Product E-commerce Customer Service (Personal Full-Stack AI Agent Project) 2026.07 - 2026.10

Tech Stack: React / TypeScript / Vite / Tailwind CSS · FastAPI / SQLAlchemy / Alembic / SQLite / LangChain / LangGraph / OpenAI

- **Agent Orchestration:** Independently developed a decoupled front-end/back-end AI customer service Agent, using LangGraph to build a looping orchestration of model and tools (StateGraph), covering the full flow of product consultation, order inquiry, after-sales review and human handoff.
- **Streaming Chat:** Designed an SSE-based streaming chat protocol (text chunks + tool progress + session snapshot push), with serial processing per session, atomic rollback on failure, and recovery after service restart.
- **Security & Concurrency:** Order ownership security—buyer_id is injected into tools by server-side runtime configuration and cannot be overridden by the model; unauthorized queries uniformly return a non-queryable result; after-sales submissions use idempotency keys to prevent duplicates, and reviews use version numbers to control concurrent conflicts.
- **Data & Authentication:** Based on SQLAlchemy + Alembic (4 migration versions) + SQLite, persisting 7 types of tables: products / sessions / messages / orders / logistics / after-sales / users; Argon2 password hashing + short-lived Bearer token + rotatable refresh token.
- **Testing & Stability:** Wrote 148 automated backend tests (model stand-in isolation, no model quota consumed) and 3 Playwright E2E suites; tool-round limits and error isolation keep long conversations stable.

Code Maintenance Assistant Based on the Claude Code Source Architecture (Personal AI Project) 2026.01 - 2026.04

Tech Stack: Python · Subprocess · AST · Function Calling

- **Long-Range Structured Memory:** Modeled on Claude Code's hot/cold memory architecture, using two kinds of MD files—a global index + fine-grained archives; in each conversation turn, MemoryTool archives high-value information such as user corrections and anticipated future needs and synchronously updates the index; new sessions read only the lightweight index and load archive fragments on demand, saving over 50% of context usage at a retrieval cost of <5 ms.
- **On-Demand Exploration & Restricted Execution:** For complex project code maintenance, designed code exploration tools and, following the JIT (Just-in-Time) principle, invoke file-system access on demand; a Subprocess whitelist mechanism blocks unauthorized operations, closing the loop of "code exploration → problem discovery → plan generation" to ensure execution safety and process coherence.
- **Adaptive Context Engineering:** Drawing on Claude Code, designed a four-layer compression mechanism; large files use AST (Abstract Syntax Tree) to extract code skeletons; progressive handling by context token ratio—early summarization above 60%, structured archiving of non-latest conversation above 85%, full compression above 95%—reducing context redundancy while retaining key information.